4-1

Harness Engineering 핵심 원칙

학습 목차

- 하네스 엔지니어링의 등장
- 하네스 엔지니어링이란?
- Anthropic - Building Effective Agents
- SKILL.md 작성 원칙

© Scopelabs. All Rights Reserved

하네스 엔지니어링의 등장

모델 성능 중심 접근의 한계

AI 활용 초기에는 더 좋은 모델과 더 좋은 프롬프트가 핵심 경쟁력으로 여겨졌다.

- 모델 성능이 낮을 때는 추론 능력과 생성 품질 자체가 가장 큰 병목이었다.
- 좋은 프롬프트는 모델이 더 정확한 답을 하도록 만드는 직접적인 방법이었다.
- 이후에는 컨텍스트 엔지니어링을 통해 정보, 규칙, 예시를 더 잘 구성하는 방식이 중요해졌다.

> **하지만 모델이 충분히 강해진 뒤에는 병목이 모델 자체보다 모델을 운영하는 구조로 이동했다.**

장시간 자율 실행의 요구

코드 작성, 리서치, 데이터 분석, 문서화는 한 번의 답변보다 긴 실행 흐름을 요구한다.

- 중간 상태, 발전 과정에 대한 기억
- 정확한 시점에 적절한 도구를 호출
- 검증 및 실패 시 재시도
- **사용자가 계속 지시 없이 이어지는 작업 흐름**

> **질문을 잘 쓰는 문제에서, AI가 일하는 시스템을 설계하는 문제로 이동했다.**

세 가지 엔지니어링의 초점

Prompt Engineering

질문 설계

모델에게 어떤 질문을 어떤 표현으로 던질지 설계한다.

Context Engineering

정보 설계

모델이 참고할 정보, 규칙, 예시, 이전 상태를 구성한다.

Harness Engineering

운영 구조 설계

AI가 목표를 수행하고 검증하며 상태를 이어가도록 실행 환경을 만든다.

하네스 엔지니어링이란?

AI가 일하는 환경을 설계하는 관점

하네스 엔지니어링은 AI가 실제 작업을 수행할 수 있도록 실행 환경을 설계하는 접근이다.

- 모델, 도구, 메모리, 역할, 작업 흐름, 검증 규칙 등을 조율
- 모델에게 더 긴 지시문을 주는 것이 아닌, 일이 끝나는 조건 제시

> **좋은 하네스는 AI를 답변기에서 실행 주체로 바꾼다.**

하네스는 위임을 위한 제어 구조

하네스는 AI를 무제한 자동화하는 장치가 아니라, 안정적으로 위임하기 위한 구조이다.

- 목표와 제약 조건을 명확히 한다.
- 사용할 수 있는 도구와 금지 행동을 구분한다.
- 중간 결과와 최종 결과를 검증할 기준을 제공한다.
- 실패했을 때 멈출지, 재시도할지, 사람에게 물을지 결정한다.

하네스 설계의 기본 순서

1. 목표 정의

완료 조건과 제약 조건을 먼저 정해 Agent가 어디까지 수행해야 하는지 알 수 있게 한다.

2. 작업 분해

큰 목표를 실행 가능한 작은 단위로 나누어 각 단계의 입력과 출력을 분명하게 만든다.

3. 도구 연결

각 단계에서 필요한 도구와 정보를 연결해 Agent가 실제 행동을 수행할 수 있게 한다.

4. 검증 설계

중간 결과와 최종 결과를 판단할 기준을 두어 작업 품질을 확인할 수 있게 한다.

5. 예외 처리

실패했을 때 멈출지, 재시도할지, 사람에게 물을지에 대한 흐름을 미리 정한다.

Anthropic - Building Effective Agents

핵심 메시지: 단순하게 시작하라

Anthropic의 `Building Effective Agents`는 복잡한 프레임워크보다 단순하고 조합 가능한 패턴을 강조한다.

- 먼저 단순한 프롬프트, 검색, 단일 도구 호출로 해결 가능한지 확인한다.
- 복잡한 구조는 명확한 성능 개선이 있을 때만 추가한다.
- 성공은 화려한 구조보다 명확한 작업 정의, 좋은 도구 설계, 피드백 루프에 달려 있다.

> **하네스는 복잡해야 좋은 것이 아니라, 문제에 맞게 충분해야 좋다.**

Workflow vs Agent (1)

Workflow는 실행 경로가 비교적 명확하게 정해진 구조이다.

- 개발자가 단계, 분기 조건, 도구 호출 순서를 미리 설계한다.
- LLM은 각 단계에서 분류, 요약, 추출, 생성 같은 제한된 역할을 수행한다.
- 예측 가능성이 높고 테스트하기 쉽다.

적합한 업무:

- 문서 요약 후 형식 변환
- 입력 유형 분류 후 전용 처리
- 코드 생성 후 테스트 실행
- 표준 템플릿 기반 산출물 작성

Workflow vs Agent (2)

Agent는 실행 경로를 모델이 상황에 따라 결정하는 구조이다.

- 목표는 주어지지만 세부 단계는 실행 중에 동적으로 정해진다.
- Agent는 관찰 결과를 보고 다음 행동을 선택한다.
- 도구 사용, 정보 수집, 계획 수정이 반복적으로 일어난다.

적합한 업무:

- 원인을 알 수 없는 오류 디버깅
- 여러 자료를 탐색하는 리서치
- 중간 판단이 필요한 모호한 작업
- 새로운 정보에 따라 계획을 바꾸는 장시간 작업

선택 기준

Workflow를 선택할 때

- 실행 순서가 명확하다.
- 분기 기준을 미리 정의할 수 있다.
- 재현성과 테스트 가능성이 중요하다.

Agent를 선택할 때

- 다음 행동을 미리 알기 어렵다.
- 환경 피드백에 따라 계획을 바꿔야 한다.
- 도구 선택과 탐색이 동적으로 필요하다.

SKILL.md 작성 원칙

Skill의 역할

`SKILL.md`는 Agent가 특정 작업을 수행할 때 참고하는 절차 문서이다.

- 언제 이 능력을 사용해야 하는지 알려준다.
- 어떻게 작업을 수행해야 하는지 알려준다.
- 프롬프트 안에 모든 지침을 넣지 않고 작업별 지침을 분리한다.
- Agent는 필요한 Skill을 선택하고 해당 절차를 따른다.

> **Skill은 하네스 안에서 Agent의 행동을 안정화하는 운영 규칙이다.**

SKILL.md의 기본 구조

상단 정보

- **Description**: 해결하는 문제와 범위
- **When to use**: 적용해야 하는 상황
- **Constraints**: 금지 행동과 안전 규칙

수행 정보

- **Workflow**: 실제 작업 순서
- **Examples**: 입력과 기대 결과
- **Verification**: 품질 기준과 확인 방법

좋은 SKILL.md의 작성 기준

좋은 Skill은 Agent가 스스로 판단하고 실행할 수 있을 만큼 구체적이어야 한다.

- 적용 조건을 모호하게 쓰지 않는다.
- 수행 절차를 실제 행동 순서대로 작성한다.
- 필요한 파일, 도구, 명령, 출력 위치를 명확히 적는다.
- 성공 조건과 검증 방법을 포함한다.
- 실패하거나 판단이 어려운 경우 어떻게 해야 하는지 제시한다.

SKILL.md 작성 순서

1. 목적

Skill이 해결하려는 작업과 적용 범위를 한 문단으로 정의한다.

2. 조건

언제 이 Skill을 사용해야 하고 언제 사용하지 않아야 하는지 구분한다.

3. 절차

Agent가 실제로 따라야 할 수행 순서를 작업 흐름에 맞게 단계별로 작성한다.

4. 도구

필요한 도구, 파일, 명령, 출력 위치를 구체적으로 명시한다.

5. 검증

최종 산출물의 품질 기준과 확인 방법을 적어 재현 가능한 검토가 가능하게 한다.

정리

> **하네스 엔지니어링은 모델을 더 잘 말하게 하는 기술이 아니라, AI가 더 안정적으로 일하게 만드는 운영 구조 설계이다.**

- 단순한 구조로 시작하고, 실패 지점을 관찰하며 확장한다.
- Workflow는 예측 가능한 절차에, Agent는 동적인 판단이 필요한 작업에 적합하다.
- `SKILL.md`는 Agent의 행동을 재현 가능하고 검증 가능한 방식으로 만드는 실천 도구이다.

`create_deep_agent` 소개

`create_deep_agent`는 하네스 엔지니어링의 구성 요소를 코드에서 조립하기 위한 함수이다.

- 모델, 도구, 시스템 프롬프트, 체크포인터, 파일 시스템 백엔드를 하나의 Agent 실행 환경으로 묶는다.
- `SKILL.md` 기반 Skill 디렉터리를 연결해 Agent가 작업별 절차를 읽고 따를 수 있게 한다.
- 도구 호출, 장시간 작업, 상태 저장, 산출물 관리를 하나의 실행 루프로 구성할 수 있다.

```
backend = FilesystemBackend(root_dir=".", virtual_mode=True)

agent = create_deep_agent(
    model=get_llm(),
    tools=[search_web],
    backend=backend,
    skills=["skills/"],
    system_prompt="범위를 벗어난 요청은 정중히 거절한다.",
    checkpointer=MemorySaver(),
)
```